

Übung 6: Prioritätswarteschlange, Heap

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, ...

1. Ternäre Heaps

Betrachten Sie die folgende Zahlenfolge:

5, 20, 10, 6, 7, 3, 1, 2

- Fügen Sie diese Zahlen in einen Min-Heap ein, bei dem jeder Knoten bis zu drei Kinder hat, statt zwei. (Anstatt in einen binären Heap, fügen wir also in einen ternären Heap ein.) Zeichnen Sie die Baumdarstellung Ihres fertigen ternären Heaps.
- Zeichnen Sie die Array-Darstellung des obigen Baums. In Ihrer Array-Darstellung sollten Sie bei Index 0 beginnen (nicht bei Index 1).
- Formulieren Sie eine Formel, um den Index des Elternteils des Knotens mit dem Index i zu finden.
- Formulieren Sie eine Formel, um das j -te Kind des Knotens mit dem Index i zu finden. Nehmen Sie an, dass $0 \leq j < 3$ ist.

2. Heaps

- Fügen Sie die folgenden Schlüssel in einen Min-Heap ein:

[10, 7, 15, 17, 12, 20, 6, 32]

- Fügen Sie nun die gleichen Schlüssel in einen Max-Heap ein.
- Fügen Sie nun die gleichen Schlüssel in einen Min-Heap ein, aber verwenden Sie den buildHeap-Algorithmus von Floyd.
- Fügen Sie [1, 0, 1, 1, 0] in einen Min-Heap ein.
- Rufen Sie removeMin für den Min-Heap auf, der als folgendes Array gespeichert ist: [2, 5, 7, 8, 10, 9]

3. Sortieren mit Heaps

- Angenommen, Sie haben eine Array-Darstellung eines Heaps. Muss das Array sortiert sein?
- Nehmen wir an, Sie haben ein sortiertes Array (in aufsteigender Reihenfolge). Muss es die Array-Darstellung eines gültigen Min-Heaps sein?
- Sie haben eine Array-Darstellung eines Min-Heaps. Wenn Sie das Array umkehren, wird es dann zu einer Array-Darstellung eines Max-Heaps?
- Beschreiben Sie den effizientesten Algorithmus, der Ihnen einfällt, um die Array-Repräsentation eines Min-Heaps in einen Max-Heap umzuwandeln. Wie lange ist seine Laufzeit?

4. Sie arbeiten an einer Anwendung zur Verwaltung von Kundenaufträgen in einem Online-Shop. Jeder Kundenauftrag hat eine eindeutige Auftragsnummer, einen Kundenname, einen Bestellwert und eine Priorität. Die Anwendung soll in der Lage sein, die folgenden Operationen effizient auszuführen:

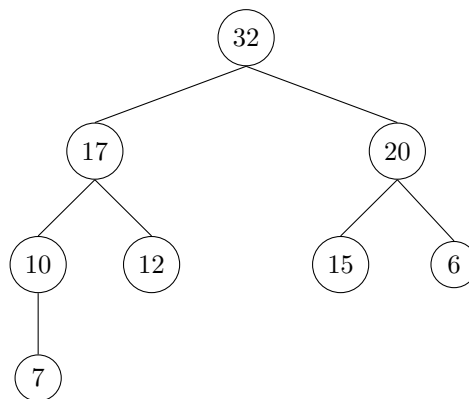
- Einfügen eines neuen Kundenauftrags mit allen seinen Daten.
- Suchen eines Kundenauftrags anhand der Auftragsnummer.
- Aktualisieren des Bestellwerts oder der Priorität eines Kundenauftrags.
- Entfernen des Kundenauftrags mit der höchsten Priorität aus der Warteschlange.

Wählen Sie einen geeigneten abstrakten Datentyp und eine Datenstruktur, um die Kundenaufträge effizient zu verwalten und die oben genannten Operationen optimal durchzuführen. Begründen Sie Ihre Wahl und erläutern Sie, warum dieser abstrakte Datentyp und diese Datenstruktur geeignet sind. Definieren Sie einen geeigneten Schlüssel zur Speicherung der Kundenaufträge. Geben Sie die O-Notation für alle vier Operationen an.

5. Sie arbeiten für eine sehr große Notaufnahme eines Krankenhauses. Normalerweise werden in einer Notaufnahme Patienten in der Reihenfolge ihres Eintreffens und des Schweregrads ihrer Verletzung behandelt. Nehmen Sie an, dass jede Patient*in als Schlüssel-Wert-Paar abgespeichert wird. Wie könnten Sie den **Schlüssel** modellieren und was könnte als **Wert** im Schlüssel-Wert-Paar abgespeichert werden? Wählen Sie dann einen abstrakten Datentyp (ADT) und eine Datenstruktur, um die Patienten im Computer der Notaufnahme in der richtigen Reihenfolge abspeichern zu können. Bitte **begründen** Sie kurz Ihre Wahl des Schlüssels. Die Wahl des ADTs und der Datenstruktur müssen Sie nicht begründen.
6. Fügen Sie die folgende Zahlenfolge in einen **Max-Heap** ein, indem Sie den buildHeap-Algorithmus von Floyd verwenden. Markieren Sie Ihre finale Lösung.

[10, 12, 20, 6, 7, 15, 17, 32]

7. Entfernen Sie das größte Element aus dem folgenden Max-Heap und zeichnen Sie den resultierenden gültigen Heap. Markieren Sie Ihre finale Lösung, falls Sie mehrere Heaps als Zwischenschritte zeichnen.



8. Ein Unternehmen möchte sicherstellen, dass seine Server regelmäßig gewartet werden. Es hat folgende Funktion `find_oldest_server` implementiert, die in einer for-Schleife alle Server durchgeht und den Server zurückgibt, der das älteste letzte Wartungsdatum hat.

Optimieren Sie die Funktion `find_oldest_server` durch die Speicherung der Server in einer geeigneten Datenstruktur und beschreiben Sie wie die neue Implementierung der Funktion `find_oldest_server` aussehen würde. Eine Beschreibung als Satz oder Pseudocode genügt. Python-Code ist alternativ auch möglich. Bestimmen Sie ebenfalls die Laufzeit der aktuellen Funktion `find_oldest_server` und die der neuen Implementierung in der O-Notation.

```
from datetime import datetime

class Server:
    def __init__(self, name, last_maintenance):
        self.name = name
        # Das Datum der letzten Wartung wird in dieser Variablen gespeichert
        self.last_maintenance = last_maintenance

    # ermöglicht Ausdrücke wie server1 < server2, wobei server1 und server2 Server Objekte sind
    def __lt__(self, other):
        return self.last_maintenance < other.last_maintenance

    def find_oldest_server(servers):
        oldest_server = servers[0]
        for server in servers:
            if server.last_maintenance < oldest_server.last_maintenance:
                oldest_server = server
        return oldest_server

# Beispiel für die Nutzung: 3 Server mit zugehörigen Datumsangaben der letzten Wartung
servers = [
    Server("Server1", datetime(2023, 5, 1)),
    Server("Server2", datetime(2023, 6, 10)),
    Server("Server3", datetime(2024, 3, 5))
]

oldest = find_oldest_server(servers)
print(f"Der Server mit dem ältesten Wartungsdatum ist: {oldest.name}")
```